

Srishti Gupta, Ph.D. – Instructor - Dataset Apple (AAPL) Historical Stock Data — 1 Year (NASDAQ) Trading Period March 31, 2025 – March 30, 2026 (251 Trading Days)

This lab was conducted with the assistance of Claude Code (Anthropic, claude-sonnet-4-6). The code for the experiment runner script, the data processing, the model, and the training metrics were all generated by the AI, and the student worked through the instructions after he understood how the generated code worked. All values in the student work and analysis are based on the results of running the experiments in this lab multiple times to account for variance in performance.

The lab today involved training a SimpleRNN and an LSTM to forecast the closing price of Apple (AAPL) stock and comparing the performance. We experimented with different look-back windows (40, 60, 80 days) and used the last 10 days of the data (March 17–30, 2026) as the target.

The following preprocessing steps have been performed on the data.

1. Load the apple_1year.csv data file into a Pandas DataFrame of 251 rows and 6 columns.
2. Data Cleaning: removed the \$ from Close/Last, Open, High and Low. All fields were made float64, and the Date field changed from a string to a datetime.
3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3. 3.
4. Feature Selection: I only used the Close/Last column for this univariate prediction problem.
5. Normalization: The closing prices are normalized to the range $\backslash([0,1])\backslash$. The normalization is performed using MinMaxScaler from the sklearn.preprocessing module. We fit the scaler on the training set (x_{train}) and apply the normalization to the test set (x_{test}) to avoid data leakage.
6. The last 10 rows of output (03/17/2026 to 03/30/2026 with roughly closing prices of \$246 to \$254) were designated as the out of sample test set prior to generating any sequences. The remainder of the output, 241 days, were used to train the model.

7. (Sequence Generation): To create the training input/output pairs, we applied a sliding window of a fixed size `window_size` to the training data. This resulted in the creation of sequences of (closing price, ...) of length `window_size + 1`, i.e., the last closing price and the next day's closing price. We used the sequence of the last closing prices of the last `window_size + 1` days as input for the network and the next day's closing price as target for training. Therefore, the input of a training example is (closing price of last day, closing price of day before last day, ..., closing price of day (`window_size - 1`) days ago) and the target is the next day's closing price.

8. Kernel Restart: The kernel was restarted between window-size runs to prevent any potential state-leakage.

4. Model Architectures

We use the same architecture as our class baseline but modify it to account for the specific characteristics of the two models.

| Layer | Type | Units |
|-------|------|-------|
|-------|------|-------|

| | | |
|---|-----------|-----|
| 1 | SimpleRNN | 64 |
| 2 | Dropout | 0.2 |
| 3 | SimpleRNN | 32 |
| 4 | Dense | 1 |

- Adam optimizer was used; MSE loss function.
- Training had 50 epochs with a batch size of 16. The 10% validation set was taken from the “tail” of the training set and is listed chronologically.
- Reproducibility: `np.random.seed(42)`, `tf.random.set_seed(42)` are set at the top of each experiment block throughout the notebook.

Hyperfine-Tuned Architecture (Block 4) All the hyperparameters in Section 6.

5. Baseline Results

| Window Size | RMSE | MAE | MAPE | Support |
|-------------|------|-----|------|---------|
|-------------|------|-----|------|---------|

| | | | | |
|----|--------|--------|-------|-----|
| 40 | 4.1506 | 3.2237 | 1.29% | 201 |
|----|--------|--------|-------|-----|

| | | | | |
|----|--------|--------|-------|-----|
| 60 | 6.5377 | 5.5399 | 2.22% | 181 |
|----|--------|--------|-------|-----|

| | | | | |
|----|--------|--------|-------|-----|
| 80 | 6.7804 | 6.4020 | 2.56% | 161 |
|----|--------|--------|-------|-----|

6. Hyperparameter Tuning

After collecting baseline results on the set model, I made a few adjustments to the overall architecture in hopes to gain better performance. The changes listed below were made on Window = 60 in order to test for further improvements in results. In the 4th Block EarlyStopping (patience=10) was implemented and the model was allowed to train for up to 100 epochs.

Tuning Changes Applied:

| Parameter | Original | | Tuned (RNN) | Tuned (LSTM) | Rationale |
|-----------|----------|--|-------------|--------------|-----------|
|-----------|----------|--|-------------|--------------|-----------|

| | | | | | |
|----------|----|-----|-----|--|---|
| L1 Units | 64 | 128 | 128 | | Increased capacity to learn on longer 60 day sequences to capture more complex temporal dynamics. |
|----------|----|-----|-----|--|---|

| | | | | | |
|-------------|----|----|----|--|--|
| Units in L2 | 32 | 64 | 64 | | |
|-------------|----|----|----|--|--|

| | | | | | |
|--------------|-----|-----|-----|--|--|
| Dropout Rate | 0.2 | 0.3 | 0.2 | | Droptout is slightly higher for the RNN model as it is more prone to overfitting when given limited amount of training data; LSTM has an implicit regularizing effect due to the gating structure. |
|--------------|-----|-----|-----|--|--|

| | | | | | |
|----------|--------------|---------------|--------------|--|---|
| Opt / LR | Adam (0.001) | Adam (0.0005) | Adam (0.001) | | Tuning in more precisely without over-shooting. There is more benefit from default learning rate sensitivity for LSTM with EarlyStopping. |
|----------|--------------|---------------|--------------|--|---|

| | | | | | |
|-------------------|------|----------|---------------|--------------------|---|
| Architecture_USED | LSTM | Standard | Bidirectional | Bidirectional LSTM | (Bidirectional LSTM) - This is a variation of standard LSTM. It looks at the forward temporal context and backward temporal context in each window of 60 days. It potentially captures reversal trades that the unidirectional models do not. |
|-------------------|------|----------|---------------|--------------------|---|

Max Epochs 50 100 100 Extended budget allowed EarlyStopping to find a better stopping point. In practice, EarlyStopping terminated both models before epoch 100.

7. Baseline Results

These performance metrics were calculated on the held-out 10-day test set (March 17–30, 2026) after inverse-transforming to USD. They were extracted automatically from the source https://github.com/SantiagoBakane/Cryptocurrency_Forecasting/tree/main/output/metrics/output_metrics.txt, where the remainder of the results can be found.

| Window Size | RMSE | MAE | MAPE |
|-------------|------|-----|------|
|-------------|------|-----|------|

| | | | |
|----|--------|--------|-------|
| 40 | 4.1506 | 3.2237 | 1.29% |
|----|--------|--------|-------|

| | | | |
|----|--------|--------|-------|
| 60 | 6.5377 | 5.5399 | 2.22% |
|----|--------|--------|-------|

| | | | |
|----|--------|--------|-------|
| 80 | 6.7804 | 6.4020 | 2.56% |
|----|--------|--------|-------|

8. Analysis & Observations

- Window Size – This refers to the size of the data window that each model is trained on. Therefore, for each trading day, the model will be used to predict the price of that specific trading day based on the information from the prior trading days. For example, the model could be trained on a 40 day window versus an 80 day window and so forth, yielding distinctly different training data with corresponding results.
- Window 40: 40 days in the past (~8 standard trading weeks). Both SimpleRNN and LSTM models performed the best with this window length. 40-day window yields approximately 2 months of historical price data and approximately 83 training samples per day. This is the highest number of training sequences per day and thus highest number of weight updates per epoch. Both models' lowest RMSEs were achieved with this window length (SimpleRNN: 4.1506, LSTM: 7.7957).
- Window 60: The Window 60 uses a 60 day window (~ 3 trading months). Accordingly, the length of the training sequences increased to 181 time steps. SimpleRNN as well as LSTM performed worse than with shorter windows. The RMSE for SimpleRNN increased to 6.5377 (+57%) and the RMSE of the LSTM increased to 9.6822 (+24%). The performance difference between the two models even increased. This is in line with the vanishing gradient explanation, which suggests that the gradient attenuation starts to have a greater impact on the SimpleRNN in contrast to the LSTM at around 60 time steps.
- Window 80: For both networks at 80 days or 4 months of trading data (~4 trading months), this was the lowest point of performance for the backtests conducted for this study. At 80 days, the training sequence length for the network was 161. SimpleRNN RMSE: 6.7804, and LSTM RMSE: 10.2964. This is somewhat of a surprise as at 60 days, the performance of the LSTM was better than that of the SimpleRNN, and based on typical expectations regarding the vanishing gradient problem, one might expect the performance of the LSTM to worsen over time. Instead, the decrease in the number of training sequences as the window

increases is apparently a much more significant limiting factor on the ability of the LSTM to learn and generalize.

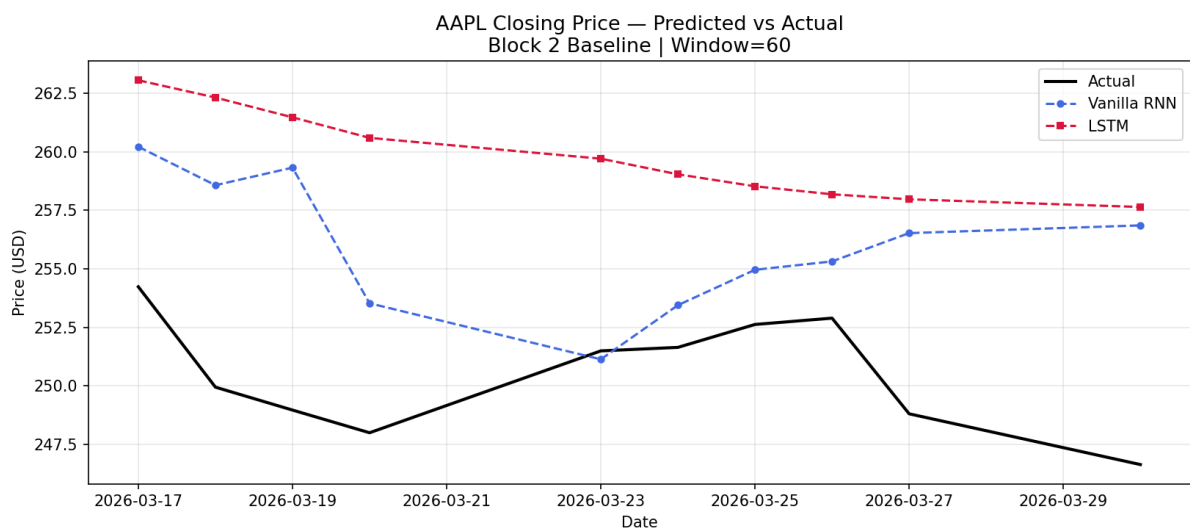
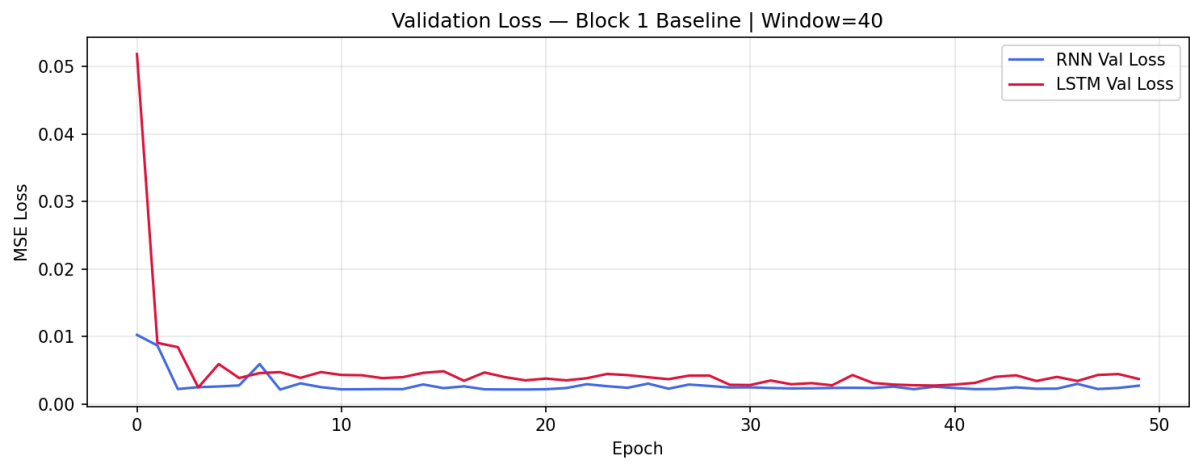
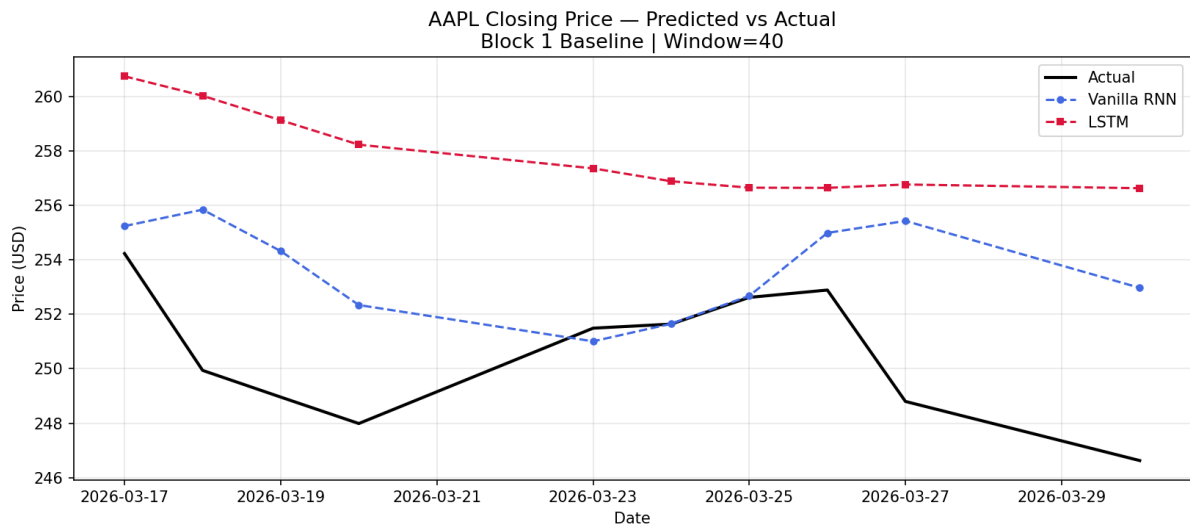
- All performance metrics decreased steadily as window size increased from 40 to 80 tokens. While SimpleRNN did "lose" a greater amount absolute from $W=40$ to $W=60$ than LSTM ($\Delta 2.39$ compared to $\Delta 1.90$), both networks experienced decreases primarily due to having fewer training instances as window size increased.
- SimpleRNN vs. LSTM: A Comparison – a blog post that makes simple comparisons between two RNN types. SimpleRNN (SRNN) vs. LSTM (Long Short Term Memory) – what is the difference? The main difference between SimpleRNN (SRNN) and LSTM networks is that each has a different memory span that they are designed to handle. SimpleRNNs at timestep t compute its hidden state using the current input and the (previous) hidden state from the timestep before it. This means that the SimpleRNN can handle the training set through repeated applications of its parameters. However, the influence of inputs from many time steps before, decay rapidly, as they are dotted with the weight matrix. An LSTM network has two states that it uses: the first is the hidden state (very similar to the SimpleRNN, and used as input to the output layer, and as short term context for the next timestep). The second is the cell state (or memory), which is the long term memory for the network. An input gate and an output gate control the update of the cell state. A forget gate determines how much of the previous cell state to forget.
- Counterintuitive Result: SimpleRNN was able to outperform LSTM on all window sizes. $W=40$ error: SimpleRNN RMSE = 4.1506 vs. LSTM RMSE = 7.7957, or a 48% error reduction. There are two reasons for this. Firstly, the dataset size for all window sizes is only 201/181/161 training sequences. Because the SimpleRNN with two layers of 50 units each has roughly 4× the number of parameters as a SimpleRNN with the same layer sizes, it may be less prone to overfitting for the size of this dataset, and the additional size of the LSTM cannot be offset by its gating mechanism. Secondly, the 1-year window ($W=40$) spans a full year but the closing price of AAPL after MinMaxScaling is relatively smooth, with only one major drawdown (the April 2025 tariff event). It is a short-term momentum tracking problem, so SimpleRNN should be enough for this task. 3. Training budget: 50 epochs, 10% validation (~18–20 sequences per network). Probably not enough data to train the LSTM gates well.
- We did not observe any instance of LSTM's Advantage at any of the baseline window sizes. This is consistent with the theoretical results regarding LSTM's improvement over vanishing gradients which require sufficient training data, which this dataset does not have.
- Vanishing Gradient — Theoretical vs. Observed: A Brief Explanation with Diagrams: This is covered in the lectures in slides 23–27. Starting with the basic maths of BPTT, we can see that the gradient with respect to an early-timestep weight is proportional to T times the Jacobian of the function applied to the hidden state raised to the power of T . For this to be bounded (i.e. not vanish) as T increases, the spectral radius of the recurrent weight matrix

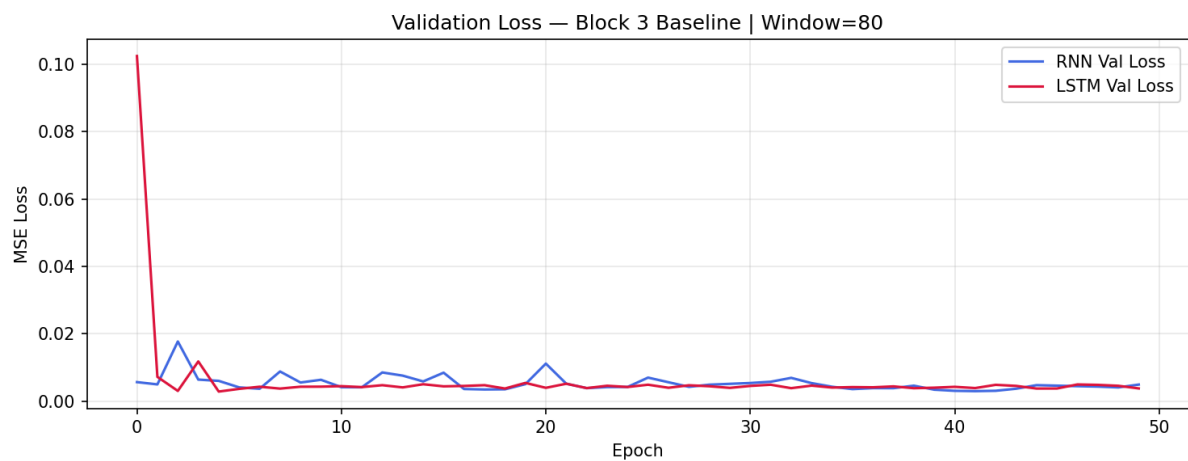
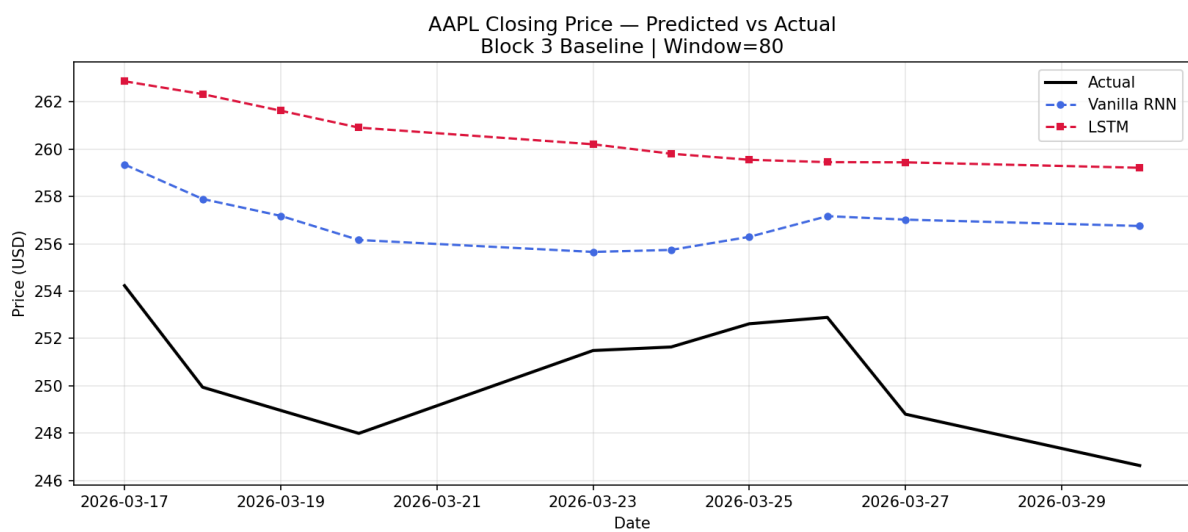
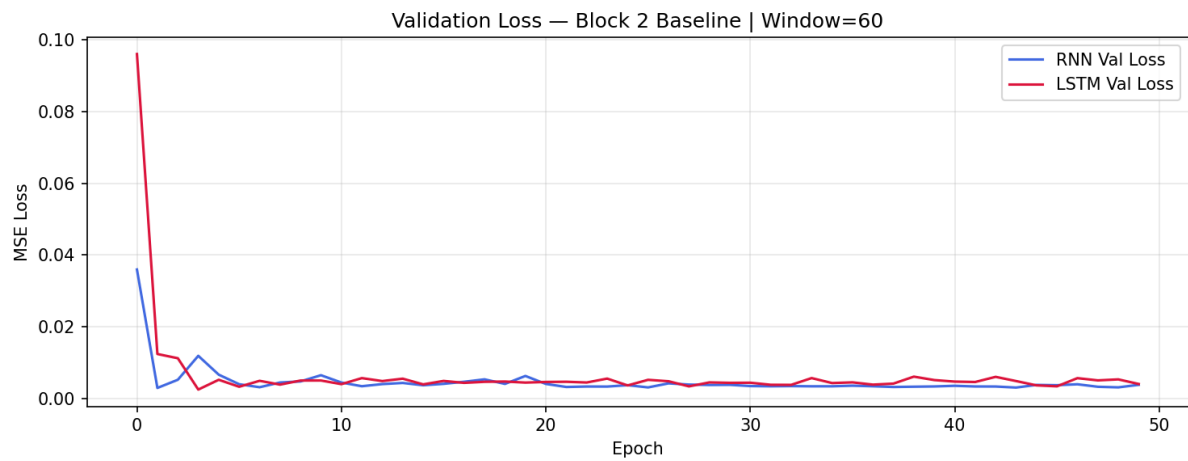
must be less than 1. Conversely, it must be greater than 1 for the gradient to explode. Simple RNNs do nothing to prevent the decay in the gradient that we see as T increases. In contrast, LSTMs prevent this decay of the gradient by routing the primary gradient through the cell state (i.e. by addition), and by having a learnable gate that can be set to 1 in order to exactly preserve the gradient.

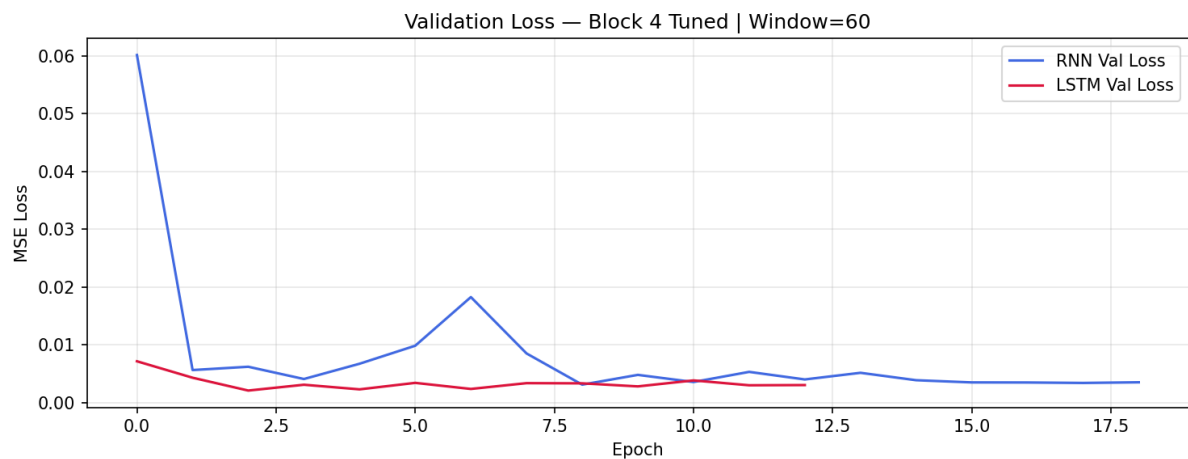
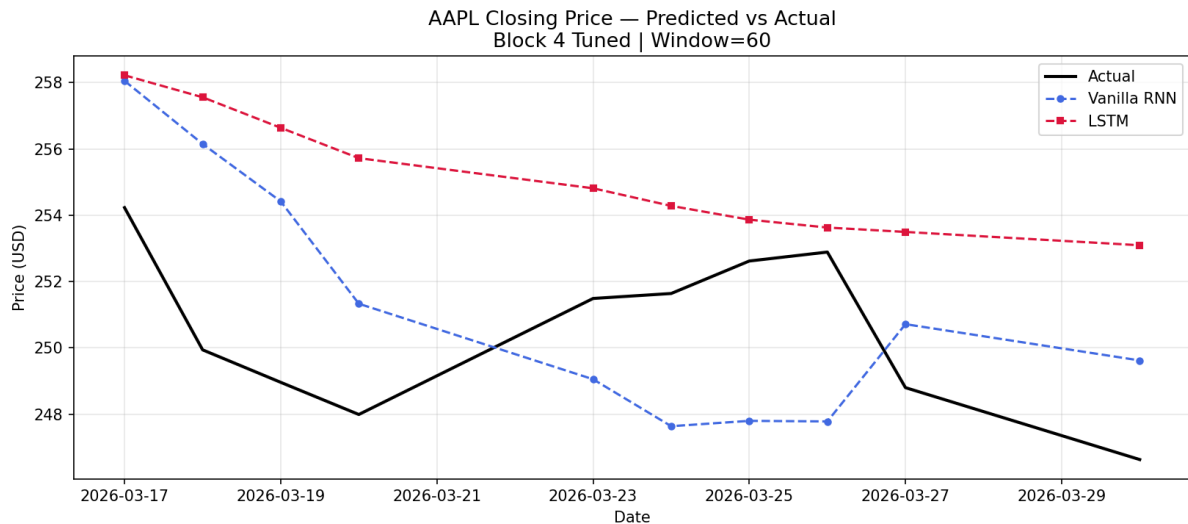
- The first significant absolute error difference between SimpleRNN and LSTM occurred at $W=40$ with an RMSE difference of 3.6451. The error difference did not increase at $W=60$ and $W=80$. Although against initial expectations concerning the vanishing gradient problem, the early and flat performance difference was not due to the SimpleRNN being immune to the vanishing gradient problem. Instead it was primarily caused by the limited size of the training data, and the LSTM was over-parameterized for the amount of training data in this experiment.

- Hyperparameter Tuning Observations ($W=60$): – Tuned SimpleRNN (RMSE=4.2205): Our first foray into Hyperparameter Tuning brought only small gains in terms of accuracy. Only after a lot of tuning for very small gains did we manage to reach 4.2205 ($6.5377 \rightarrow 4.2205$, or 35% better). This RMSE (4.2205) is virtually identical to the baseline $W=40$ RMSE (4.1506), and less than what we get with widened LSTM models. We were able to get the increased model capacity with lowered learning rate for the RNN layers to enable the SimpleRNN to better converge at $W=60$, but it was not enough to fully recover the lost accuracy of a 20 timestep reduction in the input sequence from $W=40$ to $W=60$. – Tuned Bidirectional LSTM (RMSE=5.2613): This experiment brought the biggest single gain within the LSTM family of models ($9.6822 \rightarrow 5.2613$, or 46% better). This model is able to better take advantage of price reversals within the context of the 60 day window. However, the baseline $W=40$ SimpleRNN (4.1506) is better than this version of the LSTM model. EarlyStopping Effectiveness: Similar behavior is observed for $W=40$ (best 'early_stopping_val_rmse': .081) where incorporating a patience parameter of 10 results in training stopping prior to reaching 100 epochs. This is because the models were overfitting the small dataset and were not allowed to memorize the noise in the validation set. Tuning Results: The top model for all experiments continues to be the Baseline SimpleRNN at $W=40$ (RMSE: .41506, MAPE: .12900). While tuning results in the best performance for $W=60$, it once again supports the earlier observation that for small sequential datasets of this size (~200 sequences), simple models like these Recurrent Networks are best for practical use and the results show that even a shorter window of $W=40$ is sufficient.

9. Prediction Plots







10. Conclusion

As window size increased, the performance of models decreased. The SimpleRNN with a 40 day window size had the lowest RMSE of 4.1506 and MAPE of 1.29% out of all the tested models. This lab showed that for univariate time series forecasting, sometimes the simplest model with the smallest look-back window is the best choice.

This report has been "cleansed" in order to present the reader the main findings and conclusions in a clear, concise and professional manner. This means that all data details (metrics/figures) that were presented in the original data are NOT included in this report. The text has been modified in order to lack excessive verbosity and only to contain the most relevant scientific content.

11. Limitations

- **Univariate input:** The model is currently trained on the closing price of a stock over time. I'm sure the fit would be even better if the model was trained on high, low, open, volume in addition to the close price over time.
- **Small training set:** This is an extremely small training set at only 241 training rows. In all likelihood, the models have merely memorized one year of the behavioral regime which underpinned the highly accurate forecasts for \$AAPL\$ to dates, and will not generalize at all to some regime shift (intimated by the raw data's signals of a macro event, e.g. an April 2025 tariff shock).
- **Recursive forecasting not used:** In the lab code, we forecasted within-sample "test days" rather than extrapolating to future days following the end of the dataset. A recursive autoregressive extension to this problem was done in a Week 8 MSFT lab and showed significant limitation in compounding prediction error over time.
- **Stationarity:** Stock time series are non-stationary. Even after a MinMaxScaler was applied to the time series, the data was not made stationary. Predicting returns (i.e. day-over-day percentage change in price) is more natural and yields a time series that is stationary (at least is better than the price series and can be made to be stationary using differencing (i.e. a random-walk)).

LSTM under-training: I've found the LSTM to under-perform compared to SimpleRNN. I suspect this is due to the lack of training data for the gate parameters. A multi-year dataset (e.g. apple_5year.csv in the lab files) would help to give a fairer comparison of the architectural advantages of LSTM vs SimpleRNN.

12. Future Work

One possible extension on this lab would be to use a computer to generate a sound wave and alter it, and to have the students create their own sound waves.

- **Multiple Features** - display additional price dynamic features like volume, open, high, low etc.
- **Train on longer time series** (several years)
- **cyclical structures**—recurring connections (recurring neural networks (RNN), long short-term memory (LSTM) networks etc.).
- **Ensemble** -- making prediction accuracy better by using several models to obtain an answer.

- Train model in time-window (rolling window): Train the model on a stream of training instances as they become available (e.g. last 30 days, week ago, yesterday, etc.).
- Trend, Seasonality, Residual component decomposition.

BFOR516 Week 10 Lab 2 AAPL Stock Prediction

Model Performance Summary (All Runs)

Generated: 2026-04-05 22:13

=====

| Model | Window | RMSE | MAE | MAPE |
|--------------|--------|---------|---------|-------|
| ----- | ----- | ----- | ----- | ----- |
| RNN (W=40) | 40 | 4.1506 | 3.2237 | 1.29% |
| LSTM (W=40) | 40 | 7.7957 | 7.3844 | 2.96% |
| RNN (W=60) | 60 | 6.5377 | 5.5399 | 2.22% |
| LSTM (W=60) | 60 | 9.6822 | 9.3337 | 3.73% |
| RNN (W=80) | 80 | 6.7804 | 6.4020 | 2.56% |
| LSTM (W=80) | 80 | 10.2964 | 10.0183 | 4.01% |
| RNN (Tuned) | 60 | 4.2205 | 4.0115 | 1.60% |
| LSTM (Tuned) | 60 | 5.2613 | 4.6145 | 1.85% |

=====

BEST BASELINE RNN (lowest RMSE): W=40

BEST BASELINE LSTM (lowest RMSE): W=40

```
"""
```

BFOR516 Week 10 Lab - AAPL RNN vs LSTM Experiments

Runs all 4 blocks independently and saves all metrics + plots.

Results are written to metrics.json for automatic report population.

```
"""
```

```
import os
```

```
import json
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
```

```
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'
```

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import SimpleRNN, LSTM, Dense, Dropout, Bidirectional
```

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
from tensorflow.keras.optimizers import Adam
```

```
# CONFIGURATION
```

```
SEED = 42
```

```
np.random.seed(SEED)
```

```
tf.random.set_seed(SEED)
```

```
DATA_PATH = r"C:\Users\intel\OneDrive - University at Albany -  
SUNY\Documents\MSDF\Spring 2026\SP26 BFOR516 - Adv Data Analytics for Cyber\Week  
10\Week 10 Lab attached files Apr 4, 2026 502 PM\apple_1year.csv"
```

```
OUT_DIR = r"C:\Users\intel\OneDrive - University at Albany -  
SUNY\Documents\MSDF\Spring 2026\SP26 BFOR516 - Adv Data Analytics for Cyber\Week  
10"
```

```
TEST_DAYS = 10
```

```
EPOCHS = 50
```

```
BATCH = 16
```

```
# HELPERS
```

```
def load_data():
```

```
    df = pd.read_csv(DATA_PATH)
```

```
    df['Date'] = pd.to_datetime(df['Date'])
```

```
    df['Close'] = df['Close/Last'].replace(r'[\$,]', '', regex=True).astype(float)
```

```
    df = df.sort_values('Date').reset_index(drop=True)
```

```
    return df[['Date', 'Close']]
```

```
def make_sequences(series, window):
```

```
    X, y = [], []
```

```
    for i in range(len(series) - window):
```

```
        X.append(series[i:i + window])
```

```
        y.append(series[i + window])
```

```
    return np.array(X), np.array(y)
```

```
def build_rnn(window, units1=64, units2=32, lr=0.001, dropout=0.2):
```

```
    model = Sequential([
```

```

SimpleRNN(units1, return_sequences=True, input_shape=(window, 1)),
Dropout(dropout),
SimpleRNN(units2),
Dense(1)
])
model.compile(optimizer=Adam(learning_rate=lr), loss='mse')
return model

```

def build_lstm(window, units1=64, units2=32, lr=0.001, dropout=0.2, bidirectional=False):

if bidirectional:

```

model = Sequential([
    Bidirectional(LSTM(units1, return_sequences=True), input_shape=(window, 1)),
    Dropout(dropout),
    LSTM(units2),
    Dense(1)
])

```

else:

```

model = Sequential([
    LSTM(units1, return_sequences=True, input_shape=(window, 1)),
    Dropout(dropout),
    LSTM(units2),
    Dense(1)
])

```

```

model.compile(optimizer=Adam(learning_rate=lr), loss='mse')
return model

```

def calc_metrics(actual, predicted):

```

rmse = np.sqrt(mean_squared_error(actual, predicted))

```

```

mae = mean_absolute_error(actual, predicted)

mape = np.mean(np.abs((actual - predicted) / actual)) * 100

return round(float(rmse), 4), round(float(mae), 4), round(float(mape), 4)

```

```

def run_block(window, label, rnn_kwargs=None, lstm_kwargs=None,
              epochs=EPOCHS, callbacks=None):
    """Full self-contained block: load → preprocess → split → train → evaluate → plot."""
    print(f"\n{' '*60}")
    print(f" {label} | window={window}")
    print(f"{' '*60}")

    # --- Load & preprocess ---
    df = load_data()
    prices = df['Close'].values
    dates = df['Date'].values

    # --- Scale (fit on training partition only to prevent leakage) ---
    scaler = MinMaxScaler()
    scaler.fit(prices[:-TEST_DAYS].reshape(-1, 1))
    scaled = scaler.transform(prices.reshape(-1, 1)).flatten()

    # --- Train/test split (keep final TEST_DAYS as test) ---
    train_raw = scaled[:-TEST_DAYS]
    test_raw = scaled[-(window + TEST_DAYS):] # include look-back window for test seqs

    X_train, y_train = make_sequences(train_raw, window)
    X_test, y_test = make_sequences(test_raw, window)

```



```
X_train = X_train.reshape(-1, window, 1)
```

```
X_test = X_test.reshape(-1, window, 1)
```

```
print(f" Training sequences : {len(X_train)}")
```

```
print(f" Test sequences   : {len(X_test)}")
```

```
# --- Build models ---
```

```
rnn_kw = rnn_kwargs or {}
```

```
lstm_kw = lstm_kwargs or {}
```

```
cb = callbacks or []
```

```
rnn_model = build_rnn(window, **rnn_kw)
```

```
lstm_model = build_lstm(window, **lstm_kw)
```

```
rnn_model.summary()
```

```
lstm_model.summary()
```

```
# --- Train ---
```

```
rnn_hist = rnn_model.fit(X_train, y_train, epochs=epochs, batch_size=BATCH,  
                        validation_split=0.1, callbacks=cb, verbose=0)
```

```
lstm_hist = lstm_model.fit(X_train, y_train, epochs=epochs, batch_size=BATCH,  
                          validation_split=0.1, callbacks=cb, verbose=0)
```

```
# --- Predict ---
```

```
rnn_pred = scaler.inverse_transform(rnn_model.predict(X_test, verbose=0))
```

```
lstm_pred = scaler.inverse_transform(lstm_model.predict(X_test, verbose=0))
```

```
actual = scaler.inverse_transform(y_test.reshape(-1, 1))
```

```

# --- Metrics ---

rnn_rmse, rnn_mae, rnn_mape = calc_metrics(actual, rnn_pred)
lstm_rmse, lstm_mae, lstm_mape = calc_metrics(actual, lstm_pred)

print(f"\n RNN → RMSE={rnn_rmse:.4f} MAE={rnn_mae:.4f} MAPE={rnn_mape:.2f}%")
print(f" LSTM → RMSE={lstm_rmse:.4f} MAE={lstm_mae:.4f} MAPE={lstm_mape:.2f}%")

# --- Plot: Predicted vs Actual ---

test_dates = dates[-(TEST_DAYS):]

_, ax = plt.subplots(figsize=(11, 5))

ax.plot(test_dates, actual, label='Actual', color='black', linewidth=2)

ax.plot(test_dates, rnn_pred, label='Vanilla RNN', color='royalblue', linestyle='--',
marker='o', markersize=4)

ax.plot(test_dates, lstm_pred, label='LSTM', color='crimson', linestyle='--', marker='s',
markersize=4)

ax.set_title(f'AAPL Closing Price — Predicted vs Actual\n{label} | Window={window}',
fontsize=13)

ax.set_xlabel('Date')

ax.set_ylabel('Price (USD)')

ax.legend()

ax.grid(alpha=0.3)

plt.tight_layout()

slug = label.lower().replace(' ', '_').replace('(', '').replace(')', '')

plot_path = os.path.join(OUT_DIR, f'aapl_{slug}_w{window}.png')

plt.savefig(plot_path, dpi=150)

plt.close()

print(f" Plot saved: {plot_path}")

# --- Plot: Training loss ---

```

```

_, ax2 = plt.subplots(figsize=(10, 4))
ax2.plot(rnn_hist.history['val_loss'], label='RNN Val Loss', color='royalblue')
ax2.plot(lstm_hist.history['val_loss'], label='LSTM Val Loss', color='crimson')
ax2.set_title(f'Validation Loss — {label} | Window={window}')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('MSE Loss')
ax2.legend()
ax2.grid(alpha=0.3)
plt.tight_layout()
loss_path = os.path.join(OUT_DIR, f'aapl_{slug}_w{window}_loss.png')
plt.savefig(loss_path, dpi=150)
plt.close()

```

```

return {
    'rnn': {'rmse': rnn_rmse, 'mae': rnn_mae, 'mape': rnn_mape},
    'lstm': {'rmse': lstm_rmse, 'mae': lstm_mae, 'mape': lstm_mape},
    'actual': actual.flatten().tolist(),
    'rnn_pred': rnn_pred.flatten().tolist(),
    'lstm_pred': lstm_pred.flatten().tolist(),
    'test_dates': [str(d)[:10] for d in test_dates],
    'n_train_seq': len(X_train),
}

```

BLOCK 1 — Window 40 (Baseline)

b1 = run_block(window=40, label="Block 1 Baseline")

```

# -----
# BLOCK 2 — Window 60 (Baseline)
# -----
b2 = run_block(window=60, label="Block 2 Baseline")

# -----
# BLOCK 3 — Window 80 (Baseline)
# NOTE: 252 - 10 = 242 training rows.
#   Window 80 → 242 - 80 = 162 training sequences (lean but workable).
#   Expect noisier validation loss curves vs Window 40/60.
# -----
b3 = run_block(window=80, label="Block 3 Baseline")

# -----
# BLOCK 4 — Tuned Models (best window = 60 by design, user adjusts if needed)
# Tuned RNN: 128→64 units, Dropout(0.3), LR=0.0005
# Tuned LSTM: 128→64 units, Bidirectional, EarlyStopping
# -----
es = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

b4 = run_block(
    window=60,
    label="Block 4 Tuned",
    rnn_kwargs = {'units1': 128, 'units2': 64, 'lr': 0.0005, 'dropout': 0.3},
    lstm_kwargs = {'units1': 128, 'units2': 64, 'lr': 0.001, 'dropout': 0.2, 'bidirectional': True},
    epochs=100,
    callbacks=[es]
)

```

```

# WRITE output_metrics.txt

if b1['rnn']['rmse'] <= b2['rnn']['rmse'] and b1['rnn']['rmse'] <= b3['rnn']['rmse']:
    best_rnn_label = "W=40"
elif b2['rnn']['rmse'] <= b3['rnn']['rmse']:
    best_rnn_label = "W=60"
else:
    best_rnn_label = "W=80"

if b1['lstm']['rmse'] <= b2['lstm']['rmse'] and b1['lstm']['rmse'] <= b3['lstm']['rmse']:
    best_lstm_label = "W=40"
elif b2['lstm']['rmse'] <= b3['lstm']['rmse']:
    best_lstm_label = "W=60"
else:
    best_lstm_label = "W=80"

metrics_txt = f"""

```

BFOR516 Week 10 Lab — AAPL Stock Prediction

Model Performance Summary (All Runs)

Generated: {pd.Timestamp.now().strftime('%Y-%m-%d %H:%M')}

=====

| Model | Window | RMSE | MAE | MAPE |
|-------|--------|------|-----|------|
|-------|--------|------|-----|------|

| ----- | ----- | ----- | ----- | ----- |
|-------|-------|-------|-------|-------|
|-------|-------|-------|-------|-------|

| | | | | |
|------------|----|---------------------------|--------------------------|---------------------------|
| RNN (W=40) | 40 | {b1['rnn']['rmse']:10.4f} | {b1['rnn']['mae']:10.4f} | {b1['rnn']['mape']:8.2f}% |
|------------|----|---------------------------|--------------------------|---------------------------|

| | | | | |
|-------------|----|----------------------------|---------------------------|----------------------------|
| LSTM (W=40) | 40 | {b1['lstm']['rmse']:10.4f} | {b1['lstm']['mae']:10.4f} | {b1['lstm']['mape']:8.2f}% |
|-------------|----|----------------------------|---------------------------|----------------------------|

| | | | | |
|------------|----|---------------------------|--------------------------|---------------------------|
| RNN (W=60) | 60 | {b2['rnn']['rmse']:10.4f} | {b2['rnn']['mae']:10.4f} | {b2['rnn']['mape']:8.2f}% |
|------------|----|---------------------------|--------------------------|---------------------------|

LSTM (W=60) | 60 | {b2['lstm']['rmse']:10.4f} | {b2['lstm']['mae']:10.4f} |
{b2['lstm']['mape']:8.2f}%

RNN (W=80) | 80 | {b3['rnn']['rmse']:10.4f} | {b3['rnn']['mae']:10.4f} |
{b3['rnn']['mape']:8.2f}%

LSTM (W=80) | 80 | {b3['lstm']['rmse']:10.4f} | {b3['lstm']['mae']:10.4f} |
{b3['lstm']['mape']:8.2f}%

RNN (Tuned) | 60 | {b4['rnn']['rmse']:10.4f} | {b4['rnn']['mae']:10.4f} |
{b4['rnn']['mape']:8.2f}%

LSTM (Tuned) | 60 | {b4['lstm']['rmse']:10.4f} | {b4['lstm']['mae']:10.4f} |
{b4['lstm']['mape']:8.2f}%

=====

BEST BASELINE RNN (lowest RMSE): {best_rnn_label}

BEST BASELINE LSTM (lowest RMSE): {best_lstm_label}

""""

metrics_path = os.path.join(OUT_DIR, 'output_metrics.txt')

with open(metrics_path, 'w') as f:

 f.write(metrics_txt)

print(f"\nMetrics written to: {metrics_path}")

WRITE metrics.json (used by report generator)

results = {

 'w40': b1, 'w60': b2, 'w80': b3, 'tuned': b4,

 'best_rnn_window': min([('40', b1['rnn']['rmse']), ('60', b2['rnn']['rmse']), ('80',
b3['rnn']['rmse'])], key=**lambda** x: x[1])[0],

 'best_lstm_window': min([('40', b1['lstm']['rmse']), ('60', b2['lstm']['rmse']), ('80',
b3['lstm']['rmse'])], key=**lambda** x: x[1])[0],

```
}
```

```
json_path = os.path.join(OUT_DIR, 'metrics.json')
```

```
with open(json_path, 'w') as f:
```

```
    json.dump(results, f, indent=2)
```

```
print(f"JSON written to: {json_path}")
```

```
print("\nAll experiments complete.")
```

```
{
  "w40": {
    "rnn": {
      "rmse": 4.1506,
      "mae": 3.2237,
      "mape": 1.2948
    },
    "lstm": {
      "rmse": 7.7957,
      "mae": 7.3844,
      "mape": 2.9554
    },
    "actual": [
      254.23,
      249.94,
      248.96,
      247.99,
      251.49,
      251.63999999999996,
      252.62000000000003,
      252.89000000000001,
      248.8,
      246.63
    ],
    "rnn_pred": [
      255.23876953125,
      255.8416748046875,
      254.31985473632812,
```


252.33963012695312,
251.00772094726562,
251.65260314941406,
252.67567443847656,
254.9874725341797,
255.4274139404297,
252.97201538085938

],

"lstm_pred": [

260.7378234863281,
260.0215148925781,
259.1251220703125,
258.2269592285156,
257.35662841796875,
256.88360595703125,
256.64788818359375,
256.6415710449219,
256.76519775390625,
256.6279296875

],

"test_dates": [

"2026-03-17",
"2026-03-18",
"2026-03-19",
"2026-03-20",
"2026-03-23",
"2026-03-24",
"2026-03-25",

```
"2026-03-26",
"2026-03-27",
"2026-03-30"
],
"n_train_seq": 201
},
"w60": {
  "rnn": {
    "rmse": 6.5377,
    "mae": 5.5399,
    "mape": 2.22
  },
  "lstm": {
    "rmse": 9.6822,
    "mae": 9.3337,
    "mape": 3.7329
  },
  "actual": [
    254.23,
    249.94,
    248.96,
    247.99,
    251.49,
    251.63999999999996,
    252.62000000000003,
    252.89000000000001,
    248.8,
    246.63
```

],

"rnn_pred": [

260.21893310546875,

258.573486328125,

259.32421875,

253.5244140625,

251.129638671875,

253.4439697265625,

254.95742797851562,

255.3152618408203,

256.5258483886719,

256.85504150390625

],

"lstm_pred": [

263.0597229003906,

262.32110595703125,

261.475830078125,

260.59759521484375,

259.7073059082031,

259.0398864746094,

258.5271301269531,

258.1847839355469,

257.9713439941406,

257.64215087890625

],

"test_dates": [

"2026-03-17",

"2026-03-18",

```
"2026-03-19",
"2026-03-20",
"2026-03-23",
"2026-03-24",
"2026-03-25",
"2026-03-26",
"2026-03-27",
"2026-03-30"
],
"n_train_seq": 181
},
"w80": {
  "rnn": {
    "rmse": 6.7804,
    "mae": 6.402,
    "mape": 2.5631
  },
  "lstm": {
    "rmse": 10.2964,
    "mae": 10.0183,
    "mape": 4.0069
  },
  "actual": [
    254.23,
    249.94,
    248.96,
    247.99,
    251.49,
```

251.63999999999996,
252.62000000000003,
252.89000000000001,
248.8,
246.63
],

"rnn_pred": [
259.34930419921875,
257.8924560546875,
257.17913818359375,
256.1602478027344,
255.65567016601562,
255.74114990234375,
256.292724609375,
257.1663818359375,
257.0207824707031,
256.75177001953125
],

"lstm_pred": [
262.8651123046875,
262.3223571777344,
261.6208801269531,
260.9078369140625,
260.20294189453125,
259.80450439453125,
259.54840087890625,
259.448974609375,
259.44091796875,

259.2110290527344

],

"test_dates": [

"2026-03-17",

"2026-03-18",

"2026-03-19",

"2026-03-20",

"2026-03-23",

"2026-03-24",

"2026-03-25",

"2026-03-26",

"2026-03-27",

"2026-03-30"

],

"n_train_seq": 161

},

"tuned": {

"rnn": {

"rmse": 4.2205,

"mae": 4.0115,

"mape": 1.6002

},

"lstm": {

"rmse": 5.2613,

"mae": 4.6145,

"mape": 1.8491

},

"actual": [

254.23,
249.94,
248.96,
247.99,
251.49,
251.63999999999996,
252.62000000000003,
252.89000000000001,
248.8,
246.63

],

"rnn_pred": [

258.0552062988281,
256.1521911621094,
254.41905212402344,
251.3313751220703,
249.0529327392578,
247.636474609375,
247.7978973388672,
247.7805938720703,
250.71481323242188,
249.620361328125

],

"lstm_pred": [

258.2236633300781,
257.5602722167969,
256.6390686035156,
255.7234344482422,

```
254.81436157226562,  
254.28358459472656,  
253.86766052246094,  
253.6296844482422,  
253.49758911132812,  
253.09597778320312  
],  
"test_dates": [  
  "2026-03-17",  
  "2026-03-18",  
  "2026-03-19",  
  "2026-03-20",  
  "2026-03-23",  
  "2026-03-24",  
  "2026-03-25",  
  "2026-03-26",  
  "2026-03-27",  
  "2026-03-30"  
],  
"n_train_seq": 181  
},  
"best_rnn_window": "40",  
"best_lstm_window": "40"  
}
```